

NumPy, Meson and f2py

Rohit Goswami · 2021-09-23 · python · fortran · numpy · f2py · ramblings

Exploring `meson` for interfacing `fortran` and `python` via `f2py` and standard techniques, with benchmarks.

Background

A [recent post](#) gauging community interest in `f2py` brought to light (among other aspects) the fact that the build systems of `f2py` are rather opaque to the end user. There are good reasons for this, since many of the tools discussed in this post were not around / in any shape to be used during the active development of `f2py` [^fn:1]. Nevertheless, build systems have come a long way from the early 2000s and the next two decades of `f2py`, will certainly involve less dependence on the soon-to-be-gone `numpy.distutils` approach to building extension modules.

The Fibonacci example

Though rather contrived and overused[^fn:2], for the rest of this post I will assume the following Fortran program, in a variety of guises.

```
C FILE: FIB1.F
      SUBROUTINE FIB(A,N)
C
C   CALCULATE FIRST N FIBONACCI NUMBERS
C
      INTEGER N
      REAL*8 A(N)
      DO I=1,N
        IF (I.EQ.1) THEN
          A(I) = 0.0D0
        ELSEIF (I.EQ.2) THEN
          A(I) = 1.0D0
        ELSE
          A(I) = A(I-1) + A(I-2)
        ENDIF
      ENDDO
      END
C END FILE FIB1.F
```

Keeping in mind the fact that the fixed form syntax is deprecated, we will also consider the more standard compliant and modern form:

```

module fib1
  implicit none
  contains
    subroutine fib(a,n)
      integer, intent(in) :: n
      integer :: i
      real(8) :: a(n)
      do i=1, n
        if (i==1) then
          a(i) = 0.0d0
        else if (i==2) then
          a(i) = 1.0d0
        else
          a(i) = a(i-1) + a(i-2)
        end if
      end do
    end subroutine
end module fib1

```

Note that this program in its current form is not really an executable, as it is essentially a subprogram; and needs to be coupled to a `main` program to work from `fortran`. This does not concern us really since we will be primarily using `python` to drive user interaction. However, for completeness (`gfortran main.f fib1.f`):

```

PROGRAM MAIN
  INTEGER, PARAMETER :: N=7
  REAL(8) :: A(N)
  CALL FIB(A,N)
  PRINT*, A
END PROGRAM

```

Along with the non-scream-case version (`gfortran main.f90 fib1.f90`):

```

program main
  use fib1, only: fib
  implicit none
  integer, parameter :: n = 7
  real(8) :: a(n)
  call fib(a,n)
  print*, a
end program main

```

NumPy Distutils

It would be remiss to not cover the most basic example of working with this. We can compile and try this out with the following:

```

f2py -m fib -c fib1.f
python -c "import fib; import numpy as np; a=np.zeros(7); fib.fib(a); print(a); exit();"

```

This is good enough, and there are standard extensions to expose more information to the binding, detailed [in the documentation](#). That, however isn't the point. The `-c` flag, convenient in many ways though it is, is a proxy for a rather more involved set of files.

```

mkdir blah
f2py -m fib -c fib1.f --build-dir blah
tree blah
blah
├── blah
│   ├── src.macosx-10.9-x86_64-3.9
│   │   ├── blah
│   │   │   ├── src.macosx-10.9-x86_64-3.9
│   │   │   ├── fortranobject.o
│   │   │   └── fortranobject.o.d
│   │   ├── fibmodule.o
│   │   └── fibmodule.o.d
├── fib1.o
└── src.macosx-10.9-x86_64-3.9
    ├── blah
    │   ├── src.macosx-10.9-x86_64-3.9
    │   ├── fortranobject.c
    │   └── fortranobject.h
    └── fibmodule.c

```

7 directories, 8 files

Indeed, the standard usage is to use a `setup.py` :

```

from numpy.distutils.core import Extension, setup
fibby = Extension(name = 'fib',
                  sources = ['fib1.f'])
if __name__ == "__main__":
    setup(name = 'fib', ext_modules = [ fibby ])

```

Which can then be built simply with:

```

python setup.py build
ag -g .so
# build/lib.macosx-10.9-x86_64-3.9/fib.cpython-39-darwin.so

```

However this does not scale very well with more general build systems, though a custom command approach for `CMake` and `scikit-build` is a [nice workaround by Nick Wogan](#).

Meson and F2PY

As `scipy` [moves towards meson](#) mainly driven by [Ralf Gommers](#), it makes sense to look into its use to drive the compilation of an extension module as well. We start by generating the files for the interface.

```

f2py fib1.f -m fib
Reading fortran codes...
    Reading file 'fib1.f' (format:fix,strict)
Post-processing...
    Block: fib
                                Block: FIB
Post-processing (stage 2)...
Building modules...
    Building module "fib"...
        Constructing wrapper function "FIB"...
            FIB(A,[N])
    Wrote C/API module "fib" to file "./fibmodule.c"

```

Before we can compile this with `meson` we need to make a few changes:

- Replace `FIB` with `fib` in `fibmodule.c` (since `C` is case-sensitive)
- Copy `f2py`'s Fortran-C-Python translation unit and header files `fortranobject.{c,h}`

With these in place, we can now setup our `meson.build`.

```
project('test_builds', 'c',
        version : '0.1',
        default_options : ['warning_level=3'])

add_languages('fortran')

# https://mesonbuild.com/Python-module.html
# requires atleast 0.46.0
py_mod = import('python')
py3 = py_mod.find_installation()
py3_dep = py3.dependency()

incdir_numpy = run_command(py3,
    ['-c', 'import os; os.chdir(".."); import numpy; print(numpy.get_include())'],
    check : true
).stdout().strip()

inc_np = include_directories(incdir_numpy)

py3.extension_module('fib1',
    'fib1.f',
    'fib1module.c',
    'fortranobject.c',
    include_directories: inc_np,
    dependencies : py3_dep,
    install : true)
```

Now we're all set.

```
meson setup builddir
meson compile -C builddir
cd builddir
python -c "import fib1; import numpy as np; a=np.empty(7); fib1.fib(a); print(a); exit();"
```

This makes for a far more pleasant experience, since `meson` works well with larger projects as well. However, note the sizes (and complexities) of the `C` portions.

```
wc -l fortranobject.c fortranobject.h fibmodule.c
1107 fortranobject.c
132 fortranobject.h
372 fibmodule.c
1611 total
```

Not something we'd probably like to mess around with. Standardization reduces the burden of user-code, as we will see in the next section.

Standard Bindings

The standard compliant approach was aptly covered almost a decade ago by [Ondřej Čertík](#) and codified in his [fortran90 best practices guide](#). We start by updating our free-form code to be compliant with newer `ISO_C_BINDING` guidelines.

```

module fib1
  use iso_c_binding
  implicit none
  contains
    subroutine fib(a,n) bind(c,name='c_fib')
      integer(c_int), intent(in), value :: n
      integer(c_int) :: i
      real(c_double) :: a(n)
      do i=1, n
        if (i==1) then
          a(i) = 0.0d0
        else if (i==2) then
          a(i) = 1.0d0
        else
          a(i) = a(i-1) + a(i-2)
        end if
      end do
    end subroutine
end module fib1

```

The main differences are the `c_type` declarations and the `bind(c,name='blah')` attribute. Also, to reduce the number of pointers for easy to copy types, we used `value`; which causes behavior similar to the pass-by-value model. With this, the a `C` driver can be as simple as:

```

/* fibby.c */
#include<stdlib.h>
#include<stdio.h>

void c_fib(double *a, int n);

int main(int argc, char* argv[argc+1]) {
  puts("Fortran fib from C:");
  int n=7;
  double a[n];
  c_fib(&a, n);
  for (int i=0; i<n; i++) {
    printf("%f ",a[i]);
  };
  return EXIT_SUCCESS;
}

```

This can now be built via:

```

gfortran -c fib1.f90
gcc fibby.c fib1.o

```

ctypes Wrapper

To wrap this over to `python` we have a few options. We can omit `main` altogether and rely on `ctypes`. So with:

```

/* cfib.c */
void c_fib(double *a, int n);

```

Followed by:

```

gfortran -c fib1.f90
gcc -fPIC -shared -o libfib.so cfib.c fib1.o

```

We can work with this in `python` as:

```
from ctypes import CDLL, POINTER, c_int, c_double
import numpy as np
fibby = CDLL("libfib.so")
a = np.zeros(7)
fibby.c_fib(a.ctypes.data_as(POINTER(c_double)), c_int(7))
print(a)
```

Which works in the same exact way as the previous approaches.

*We have gone from over a 1000 lines of `C` to **one line***^[^fn:3]

Cython wrapper

However, we can skip even the one line of `C`, by using `cython`.

```
# fibby.pyx
from numpy cimport ndarray
import numpy as np

cdef extern:
    void c_fib(double *a, int n)

def fib(double a, int n):
    cdef ndarray[double, mode="c"] ret = np.zeros(n)
    c_fib(&ret, n)
    return ret
```

This is closer to `f2py` in the sense that the `C` code is simply hidden. It requires a compilation step typically done via `setuptools.py`, but, this is much nicer with `meson` since from `0.59.0`, [Cython is supported](#).

```
project('test_builds', 'c',
        version : '0.1',
        default_options : ['warning_level=3'])

add_languages('fortran', 'cython')

py_mod = import('python')
py3 = py_mod.find_installation()
py3_dep = py3.dependency()

py3.extension_module('fibby',
    'fibby.pyx',
    dependencies : py3_dep
)
```

As before.

```
meson setup bdircython
meson compile -C bdircython
cd bdircython
python -c "import fibby; import numpy as np; a=np.empty(7); b=fibby.fib(a); print(b); exit();"
```

Note the difference in the calling semantics here. Also, recall that `.pyx` files are actually used to generate `C` code.

```
cython fibby.pyx
wc -l fibby.c
6376 fibby.c
```

This is vastly more code compared to the `f2py` generated wrapper code; which seems pretty unattractive.

Python-C API

Neither `ctypes` nor `cpython` is as friendly/clean as loading an extension module, so it makes more sense to actually write a proper `Python-C` extension module for a more fair comparison. This however, involves quite some reference counting and familiarity with the [NumPy-C API](#). Actually it is not too bad for the function we are considering. Even a well commented and naive hand crafted version clocks in below 90 lines of code.

```

#ifndef PY_SSIZE_T_CLEAN
#define PY_SSIZE_T_CLEAN
#endif /* PY_SSIZE_T_CLEAN */

#include "Python.h"
#include "numpy/ndarrayobject.h"
#include "numpy/ufuncobject.h"

static PyMethodDef FibbyMethods[] = {
    {NULL, NULL, 0, NULL}
};

void c_fib(double *a, int n);

/* The loop definition must precede the PyMODINIT_FUNC. */

static void double_fib(char **args, npy_intp *dimensions,
                       npy_intp* steps, void* data)
{
    int i; // Standard integer is fine here
    npy_intp n = dimensions[0];
    char *in = args[0], *out = args[1];
    npy_intp in_step = steps[0], out_step = steps[1];

    double apointer[n];

    for (i = 0; i < n; i++) {
        apointer[i] = (double)in[i];
    }

    // Call the Fortran function
    c_fib(apointer, n);

    for (i = 0; i < n; i++) {
        /*BEGIN main ufunc computation*/
        *((double *)out) = apointer[i];
        /*END main ufunc computation*/

        in += in_step;
        out += out_step;
    }
}

/*This a pointer to the above function*/
PyUFuncGenericFunction funcs[1] = {&double_fib};

/* These are the input and return dtypes of fib.*/
static char types[2] = {NPY_DOUBLE, NPY_DOUBLE};

static void *data[1] = {NULL};

static struct PyModuleDef moduledef = {
    PyModuleDef_HEAD_INIT,
    "fibby", /* module name */
    NULL, /* module docstring */
    -1, /* -1 means global state, no sub-interpreters */
    FibbyMethods, /* Module level functions */
    NULL, /* single phase initialization, NULL slots */
    NULL, /* traversal function for GC */
    NULL, /* clear function for the GC */
    NULL /* deallocation function for the GC */
};

PyMODINIT_FUNC PyInit_fibby(void)
{

```



```

PyObject *m, *fib, *d;
m = PyModule_Create(&moduledef);
if (!m) {
    return NULL;
}

import_array();
import_umath();

fib = PyUFunc_FromFuncAndData(funcs, data, types, 1, 1, 1,
                               PyUFunc_None, "fib",
                               "Calls the fib.f90 Fibonacci subroutine", 0);

d = PyModule_GetDict(m);

PyDict_SetItemString(d, "fib", fib);
Py_DECREF(fib);

return m;
}

```

Compiling this with `meson` is a snap.

```

project('test_builds', 'c',
        version : '0.1',
        default_options : ['warning_level=3'])

add_languages('fortran', 'cython')

py_mod = import('python')
py3 = py_mod.find_installation()
py3_dep = py3.dependency()

incdir_numpy = run_command(py3,
    ['-c', 'import os; os.chdir(".."); import numpy; print(numpy.get_include())'],
    check : true
).stdout().strip()

inc_np = include_directories(incdir_numpy)

py3.extension_module('fibby',
    'fib1.f90',
    'fibbyhand.c',
    include_directories: inc_np,
    dependencies : py3_dep
)

```

We can compile this much the same as the others.

```

meson setup bdircythonhand
meson compile -C bdircythonhand
cd bdircythonhand
python -c "import fibby; import numpy as np; a=np.empty(7); b=fibby.fib(a); print(b); exit();"

```

This has the lowest lines of code too.

```

wc -l fibbyhand.c
85 fibbyhand.c

```

It is rather remarkable since the `C` code here is not optimized in the least. The twin `for` loops can almost certainly be handled more efficiently, and also from the point of view of the Fortran-C interface too, this is lacking. Nevertheless, it will suffice for our purpose here, which has more to do with wrapping automatically (and therefore rather naively) than generating optimal code (but that would be great).

Benchmarks

We can use [hyperfine](#) for basic benchmarks.

```
hyperfine 'python -c "import fibby; import numpy as np; a=np.empty(7); b=fibby.fib(a); print(b); exit();"' --warmup 5 -s full -r 25
Benchmark #1: python -c "import fibby; import numpy as np; a=np.empty(7); b=fibby.fib(a); print(b); exit();"
  Time (mean ± σ):      155.4 ms ±   9.7 ms    [User: 133.1 ms, System: 40.2 ms]
   Range (min ... max):  145.6 ms ... 190.4 ms    25 runs
```

Results for the various approaches are collected below in Table [tbl:bench](#).

Table 1: The results are for 25 runs with 5 warmup for `np.empty(7)` as the input

Command	Mean [ms]	Min [ms]	Max [ms]
Handwritten NumPy-C-Fortran	126.0 ± 3.9	119.8	136.8
F2PY (F77)	129.1 ± 4.0	125.1	140.4
Cython	129.5 ± 6.8	121.4	149.1
F2PY (F90)	129.9 ± 5.1	123.9	145.8
ctypes	128.3 ± 7.8	122.7	159.8

As expected, `ctypes` is the slowest, since it requires the most massaging of inputs on the Python side. Surprisingly, loading the free form file with `f2py` was quite a bit slower than the `f77` file. Given the complexity of the files involved, it is actually surprising that the handwritten wrapper shows a rather clear gain in performance; given its implementation; but nonetheless it supports the hypothesis that advances in the NumPy-C API and the Fortran-C interoperability has improved significantly over time.

Conclusions

`f2py`, like `Fortran`, is here to stay. As the standards strive towards better compliance with `C` however, it also gets larger and rather more complicated [^fn:4]. One thing which was not covered here was the actual `ISO_Fortran_binding.h` [^fn:5] and other modifications to the `C` code while interpolating with Fortran itself.

To hold its place in the larger ecosystem, `f2py` will be essential in protecting the average user from the standard itself, automatically providing optimal bindings for users as initially planned by [Pearu Peterson](#). Modernization efforts with [Melissa Weber Mendonça](#), Ralf, Pearu, and others at Quansight (including the rest of the community and [me](#)) are sure to gravitate towards generating better, more reliable interfaces to bridge the gaps between Fortran and Python as supported by the NumPy core developers [^fn:6]. Clearly, the way forward is to work in lockstep with the NumPy-C API and recent Fortran standards.